

Section 1 - Project Description

1.1 Project SÜPRÜZ

1.2 Description

Süprüz is a mobile-based platform designed to reduce food waste by connecting food businesses with consumers seeking affordable meals. Businesses can list their unsold but still fresh products as discounted "Surprise Packages" at any time during their operating hours. Consumers discover and reserve these packages through a location-aware mobile application, and complete the transaction in-store via QR code verification. The platform consists of three core components: a consumer-facing mobile application (iOS & Android), a business partner portal (mobile & web), and a centralized cloud-based backend. Its core differentiator is a proximity-based reservation engine that requires consumers to be physically within a 5 KM radius of a business in order to make a reservation, driving immediate local foot traffic and reducing no-shows.

1.3 Revision History

Date	Comment	Author
12/04/2026	Template has been created	Emre
04/05/2026	Section 3 & Section 8 - System Architecture thoroughly designed + Outer Interfaces(Dependencies)	Emre
04/05/2026	Completed Section 4 (Data Dictionary) and Section 6 (Data Design) including ER Diagram, Persistent/Transient Data, and Data Transformations.	Enes
04/05/2026	Section 1 completed	Beyza

Contents

[Section 1 - Project Description](#)

[1.1 Project](#)

[1.2 Description](#)

[1.3 Revision History](#)

[Section 2 - Overview](#)

[2.1 Purpose](#)

[2.2 Scope](#)

[2.3 Requirements](#)

[2.3.1 Estimates](#)

[2.3.2 Traceability Matrix](#)

[Section 3 - System Architecture](#)

[Section 4 - Data Dictionary](#)

[Section 5 - Software Domain Design](#)

[5.1 Software Application Domain Chart](#)

[5.2 Software Application Domain](#)

[5.2.1 Domain X](#)

[5.2.1.1 Component Y of Domain X](#)

[5.2.1.1.1 Task Z of Component Y1 of Domain X](#)

[Section 6 – Data Design](#)

[6.1 Persistent/Static Data](#)

[6.1.1 Dataset](#)

[6.1.2 Static Data](#)

[6.1.3 Persisted data](#)

[6.2 Transient/Dynamic Data](#)

[6.3 External Interface Data](#)

[6.4 Transformation of Data](#)

[Section 7 - User Interface Design](#)

[7.1 User Interface Design Overview](#)

[7.2 User Interface Navigation Flow](#)

[7.3 Use Cases / User Function Description](#)

[Section 8 - Other Interfaces](#)

[8.1 Interface X](#)

[Section 9 - Extra Design Features / Outstanding Issues](#)

[Section 10 – References](#)

[Section 11 – Glossary](#)

Section 2 - Overview

2.1 Purpose

This document describes the software design of the Süprüz platform, Version 1.0. It covers the complete system including the consumer mobile application, the business partner portal, and the cloud-based backend services. The intended audience includes software developers and engineers responsible for implementation, QA engineers deriving test cases from this design, UI/UX designers ensuring interface consistency, and project managers tracking development progress.

2.2 Scope

This document covers the full scope of the Süprüz Version 1.0 platform. The system enables food businesses to list discounted surplus food packages and allows consumers to discover, reserve, and collect them via a QR-based verification flow. The design encompasses six core modules: User Registration and Profile Management, Geolocation-Based Discovery, Proximity-Based Reservation Engine, Inventory and Package Management, QR-Based Reservation Verification, and Push Notifications.

2.3 Requirements

Your mileage may vary -- we typically break down the requirements to provide a ballpark estimate

2.3.1 Estimates

#	Description	Hrs. Est.
1	Brief description of task / module with link	# est
	TOTAL:	# est tot

2.3.2 Traceability Matrix

Cross reference this document with your requirements document and link where you satisfy each requirement

SRS Requirement	SDD Module
Req 1	5.1.1 (link to module), 5.1.2 (link)

Section 3 - System Architecture

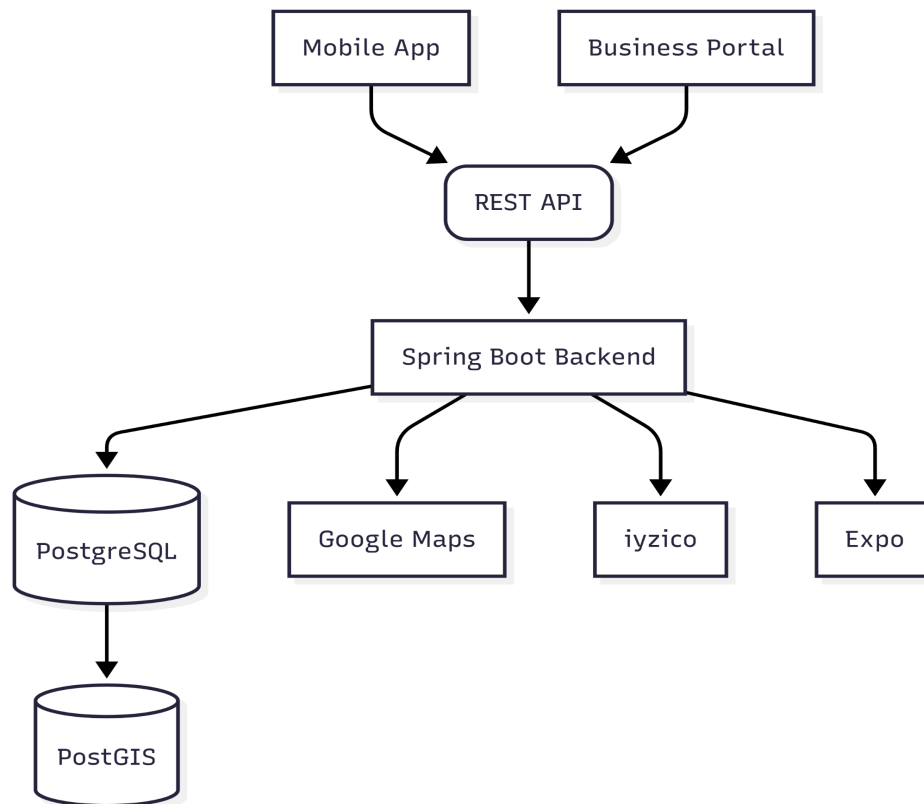
3.1 Architecture Overview

The Süprüz system follows a **Modular Monolith Architecture**, where all core business logic is centralized in a single backend service while maintaining logical modularity.

The system consists of three primary layers:

- **Client Layer**
 - Consumer Mobile Application (React Native + Expo)
 - Business Partner Portal (Mobile/Web)
 - Admin Portal (Web)
- **Application Layer (Backend)**
 - Spring Boot REST API
 - Handles authentication, reservations, QR validation, and business logic
- **Data Layer**
 - PostgreSQL database with PostGIS extension for geolocation queries
- **External Services**
 - Payment Gateway (iyzico)
 - Google Maps API (geolocation & mapping)
 - Push Notification Infrastructure (FCM/APNs via Expo)

3.2 Architecture Diagram



3.3 Data Flow Explanation

The mobile client communicates with the backend via RESTful APIs over HTTPS. The backend processes business logic and interacts with the PostgreSQL database through JPA Repository for persistent storage. External services such as Google Maps API and iyzico are invoked for geolocation and payment processing respectively.

3.4 Component Interactions and Data Flow

This section describes how the main components of the Süprüz system interact during runtime. The system follows a client-server model where all client applications communicate with the backend via RESTful APIs over HTTPS.

3.4.1 Discovery Flow

When a consumer opens the discovery screen, the mobile application retrieves the user's GPS location and sends it to the backend. The backend processes this request using spatial queries in the PostgreSQL/PostGIS database to retrieve nearby businesses within the defined radius. The results are returned to the client and rendered as map pins and list items.

3.4.2 Reservation Flow

When a user initiates a reservation, the mobile client sends a request to the backend. The backend validates the user's location (5 KM rule) and applies a pessimistic lock on the package inventory to prevent race conditions. Upon successful validation, the system creates a reservation record, updates inventory, and returns a confirmation response to the client.

3.4.3 QR Verification Flow

During in-store pickup, the consumer presents a QR code generated by the system. The business portal scans this code and sends the data to the backend. The backend validates the QR code by checking its timestamp, reservation ID, and associated business. If valid, the reservation status is updated accordingly (e.g., "Verified / Paid" or "Completed").

3.4.4 Payment Flow

For in-app payments, the backend communicates with the iyzico payment gateway via secure HTTPS requests. The payment provider processes the transaction and returns a success or failure response, which is then reflected in the system's reservation status.

3.4.5 Notification Flow

When a new package is created, the backend triggers a push notification request via Expo Notifications(FCM & APNs). The notification is delivered to eligible users based on their last known location.

Section 4 - Data Dictionary

This section defines the core database tables, fields, and data types that will be persisted within the PostgreSQL/PostGIS database.

Table 1: USERS : Stores identity and profile information for all system actors (Consumers, Business Staff, Admins).

USERS		
Field	Notes	Type
id	Primary Key. Unique Identifier (UUID)	UUID
email	User's registered email address (Must be unique)	VARCHAR(255)
oauth_subject_id	Unique identifier returned by Google/Apple SSO	VARCHAR(255)
display_name	Full name or chosen display name of the user	VARCHAR(100)
role	Defines access level: CONSUMER, STAFF, ADMIN	VARCHAR(20)
no_show_count	Tracks missed reservations (resets every 30 days)	INTEGER
is_suspended	Boolean flag indicating if the account is currently suspended	BOOLEAN
created_at	Timestamp of account creation	TIMESTAMP

Table 2: BUSINESSES : Stores vendor details and their exact geographical location for spatial queries.

BUSINESSES		
Field	Notes	Type
id	Primary Key. Unique Identifier (UUID)	UUID
name	User's registered email address (Must be unique)	VARCHAR(255)
category	Unique identifier returned by Google/Apple SSO	VARCHAR(50)
location	Full name or chosen display name of the user	GEOMETRY(Point, 4326)
address	Defines access level: CONSUMER, STAFF, ADMIN	TEXT
operating_hours	Tracks missed reservations (resets every 30 days)	JSONB
is_active	Boolean flag indicating if the account is currently suspended	BOOLEAN

Table 3: SURPRISE PACKAGES : Stores inventory listed by businesses.

SURPRISE_PACKAGES		
Field	Notes	Type
id	Primary Key. Unique Identifier (UUID)	UUID
business_id	Foreign Key linking to BUSINESSES	UUID
description	Details regarding the package contents	TEXT
original_price	Retail price before discount (₺)	DECIMAL(10,2)
discounted_price	Price after mandatory $\geq 50\%$ discount (₺)	DECIMAL(10,2)
initial_quantity	Total packages offered at time of creation	INTEGER
remaining_quantity	Packages currently available (Used for pessimistic locking)	INTEGER
pickup_start_time	Start of the valid QR scanning window	TIMESTAMP
pickup_end_time	End of the valid QR scanning window	TIMESTAMP
status	State of listing: ACTIVE, CLOSED, EXPIRED	VARCHAR(20)

Table 4: RESERVATIONS : Tracks the lifecycle of a consumer's order from creation to pickup.

RESERVATIONS		
Field	Notes	Type
id	Primary Key. Unique Identifier (UUID)	UUID
consumer_id	Foreign Key linking to USERS (Role: Consumer)	UUID
package_id	Foreign Key linking to SURPRISE PACKAGES	UUID
status	State: RESERVED, VERIFIED/PAID, COMPLETED, NO SHOW, CANCELLED	VARCHAR(30)
qr_code_token	Encrypted string representing the unique QR payload	TEXT
created_at	Timestamp of reservation creation	TIMESTAMP
fulfilled_at	Timestamp of successful QR scan/pickup	TIMESTAMP

Section 5 - Software Domain Design

5.1 Software Application Domain Chart

Süprüz has a modular structure where multiple domains work together. The main domains are:

- User Domain
- Vendor (Restaurant) Domain
- Order Domain
- Payment Domain

- Notification Domain
- Database Domain
- Location Domain

5.2 Software Application Domain

The system consists of various domains that support users in discovering, purchasing and receiving these packages. Each domain assumes a specific responsibility, ensuring the system is scalable and sustainable.

Relationships between the domains:

- User Domain → Order Domain: User places the order
- User Domain → Location Domain: User location is captured and updated for nearby service filtering
- Location Domain → Vendor Domain: Nearby restaurants are retrieved based on user location
- Location Domain → Order Domain: Location data is used to filter available packages during order creation
- Vendor Domain → Order Domain: Restaurant manages the order
- Order Domain → Payment Domain: Payment process is triggered
- Order Domain → Notification Domain: Order status is notified
- Entire Domains → Database Domain: Data storage and retrieval operations

5.2.1 Order Domain

Order Domain is the core domain of the Süprüz, responsible for managing the entire lifecycle of orders. It handles order creation, validation, integration with the payment process and tracking of order status updates.

5.2.1.1 Family of Components

- Order Service (Business Logic Layer)
- Order Controller (API Layer)
- Order Repository (Data Access Layer)
- Order Status Manager
- Concurrency Control Mechanism (to prevent stock conflicts)

5.2.1.2 Database Structure

5.2.1.3 Component Relationships

- Order Controller invokes the Order Service
- Order Service interacts with the database via Repository
- Order Service communicates with the Payment Domain
- Order Service sends events to the Notification Domain

5.2.1.4 Data Flow

- Order request is received via API
- Order Service performs validation
- Order is stored in the database
- Payment request is sent to Payment Domain

- Order status is updated based on payment result
- Notification Domain is triggered

5.2.1.5 Error Handling & Edge Cases

- Package out of stock → order rejected
- Payment failure → order marked as “failed”
- Duplicate requests → handled via idempotency

5.2.1.6 Security Considerations

- Only authenticated users can create orders
- Authorization checks ensure users access only their own orders

5.2.1.7 Scalability Considerations

- Supports concurrent order requests using locking or optimistic concurrency
- Can be scaled horizontally with stateless service design

5.2.1.8 Dependencies

- Depends on User Domain (user validation)
- Depends on Vendor Domain (package availability)
- Depends on Payment and Notification Domains

5.2.1.9 State Management

Order states :

- Pending → Paid → Completed → Cancelled → Failed
- Pending → Paid → Failed

5.2.1.10 Business Rules

- Orders can only be created for active packages
- A package cannot be oversold beyond its stock

5.2.1.11 Performance Considerations

- Frequently accessed orders can be cached

5.2.1.12 Design Considerations

- Security: authentication and authorization enforced
- Scalability: supports high concurrent usage
- Performance: optimized queries and caching

5.2.2 User Domain

The User Domain is responsible for user authentication, authorization and profile management. It plays a critical role in ensuring system security and user experience.

5.2.2.1 Family of Components

- Authentication Service
- Authorization Manager
- User Profile Service
- Session Management
- Token Generator (JWT)

5.2.2.2 Database Structure

5.2.2.3 Component Relationships

- Authentication Service interacts with the User Database
- Authorization Manager enforces role-based access control
- User Domain provides user data to Order Domain

5.2.2.4 Data Flow

- User sends login/register request
- Authentication Service validates credentials
- Token is generated
- Session is stored in the database

5.2.2.5 Error Handling & Edge Cases

- Invalid credentials → login failure
- Duplicate email → registration rejected
- Expired session → re-authentication required

5.2.2.6 Security Considerations

- Passwords are hashed before storage
- Token-based authentication (JWT)
- Role-based access control

5.2.2.7 Scalability Considerations

- Stateless authentication using tokens
- Can support large number of concurrent users

5.2.2.8 Dependencies

- Provides user data to Order and Vendor Domains

5.2.2.9 State Management

- Active / Inactive users

- Session states (valid / expired)

5.2.2.10 Business Rules

- Each email must be unique
- Password must meet security requirements

5.2.2.11 Performance Considerations

- Token validation optimized

5.2.2.12 Design Considerations

- Security-focused design
- High availability for authentication services

5.2.3 Vendor (Restaurant) Domain

The Vendor Domain enables restaurants to create, manage and monitor food packages offered on the platform.

5.2.3.1 Family of Components

- Vendor Service
- Package Management Service
- Inventory Control
- Vendor Dashboard API

5.2.3.2 Database Structure

5.2.3.3 Component Relationships

- Vendor Service interacts with Package Management Service
- Package Service provides data to Order Domain
- Inventory Control synchronizes with Order Domain

5.2.3.4 Data Flow

- Vendor creates a package
- Package is stored in the database
- Users browse available packages
- Stock is updated upon order placement

5.2.3.5 Error Handling & Edge Cases

- Invalid package data → rejected
- Expired package → automatically disabled
- Stock mismatch → corrected via sync

5.2.3.6 Security Considerations

- Only authorized vendors can manage packages

- Vendor-specific data isolation

5.2.3.7 Scalability Considerations

- Supports multiple vendors simultaneously
- Can scale package listing services independently

5.2.3.8 Dependencies

- Interacts with Order Domain (stock updates)
- Uses User Domain for vendor authentication

5.2.3.9 State Management

- Package states: Active → Expired → Sold Out

5.2.3.10 Business Rules

- Packages must have a valid time window
- Discount pricing must be lower than original price

5.2.3.11 Performance Considerations

- Frequently viewed packages cached
- Optimized queries for listing

5.2.3.12 Design Considerations

- Scalable vendor onboarding
- Real-time stock synchronization

5.2.4 Payment Domain

The Payment Domain handles all financial transactions in the system. It ensures secure payment processing, validation, and transaction tracking.

5.2.4.1 Family of Components

- Payment Service
- Payment Gateway Adapter (iyzico)
- Transaction Manager

5.2.4.2 Database Structure

5.2.4.3 Component Relationships

- Payment Service communicates with external payment gateways
- Payment Domain sends results to Order Domain
- Order Domain initiates payment
- Payment Service sends request to gateway
- Response is received
- Result is returned to Order Domain

5.2.4.4 Data Flow

- Order Domain initiates payment
- Payment Service sends request to gateway
- Response is received
- Result is returned to Order Domain

5.2.4.5 Error Handling & Edge Cases

- Payment declined → order marked failed
- Network failure → retry mechanism
- Duplicate transaction → prevented

5.2.4.6 Security Considerations

- Sensitive payment data encrypted
- Compliance with secure payment standards
- External gateways (iyzico)

5.2.4.7 Scalability Considerations

- External payment services handle load
- Asynchronous processing supported

5.2.4.8 Dependencies

- Called by Order Domain
- Communicates results back to Order Domain

5.2.4.9 State Management

- Payment states: Pending → Completed → Failed → Refunded
- Payment states: Pending → Completed

5.2.4.10 Business Rules

- Payment must be completed before order confirmation
- Refund allowed under specific conditions

5.2.4.11 Performance Considerations

- Minimize latency in payment processing
- Retry logic optimized

5.2.4.12 Design Considerations

- High security priority
- Fault-tolerant integration

5.2.5 Notification Domain

The Notification Domain is responsible for informing users and vendors about system events such

as order updates and promotions.

5.2.5.1 Family of Components

- Notification Service
- Message Queue
- Push Notification Service
- Email Service

5.2.5.2 Database Structure

5.2.5.3 Component Relationships

- Order Domain triggers Notification Domain
- Notification Service operates asynchronously via message queue

5.2.5.4 Data Flow

- Event occurs (e.g., order created)
- Notification Service generates message
- Message is sent to user/vendor

5.2.5.5 Error Handling & Edge Cases

- Failed delivery → retry
- Invalid user → discard

5.2.5.6 Security Considerations

- Prevent unauthorized notification sending
- Data privacy for messages

5.2.5.7 Scalability Considerations

- Uses asynchronous messaging
- Can handle high volume notifications

5.2.5.8 Dependencies

- Triggered by Order, Payment, and Vendor Domains

5.2.5.9 State Management

- Notification states: Pending → Sent → Failed
- Notification states: Pending → Sent → Successful

5.2.5.10 Business Rules

- Notifications triggered only on significant events
- Duplicate notifications avoided

5.2.5.11 Performance Considerations

- Asynchronous processing improves responsiveness
- Batch sending possible

5.2.5.12 Design Considerations

- Highly scalable
- Event-driven architecture

5.2.6 Database Domain

The Database Domain is responsible for persistent data storage, integrity and efficient data retrieval across the Süprüz. It serves as the foundational layer that supports all other domains by ensuring consistency, reliability, and transactional safety of data.

This domain centralizes structured data management and enforces constraints, relationships and business rules at the database level.

5.2.6.1 Family of Components

- Relational Database Management System (RDBMS) (PostgreSQL)
- Schema Definition Layer
- Transaction Management System (ACID compliance)

5.2.6.2 Database Structure

5.2.6.3 Component Relationships

- All application domains interact with the Database Domain via repositories

5.2.6.4 Data Flow

- Application domains send queries through data access layer
- Database processes requests using queries, procedures, or functions
- Triggers automatically enforce rules and updates
- Processed data is returned to application services

5.2.6.5 Transaction Management

- Ensures atomicity in operations such as order creation and payment
- Supports rollback in case of failure
- Maintains consistency across multiple related tables

5.2.6.5 Error Handling & Edge Cases

- Transaction failure → rollback
- Constraint violation → rejected operation

5.2.6.6 Security Considerations

- Data encryption (at rest / in transit)

- Access control mechanisms

5.2.6.7 Scalability Considerations

- Supports vertical and horizontal scaling
- Read replicas for high load

5.2.6.8 Dependencies

- Used by all domains

5.2.6.9 State Management

- Maintains consistent data states across transactions

5.2.6.10 Business Rules

- Enforces integrity constraints
- Ensures consistency across related data

5.2.6.11 Performance Considerations

- Query optimization
- Efficient transaction handling

5.2.6.12 Design Considerations

- High reliability and consistency
- ACID-compliant transaction support

5.2.7 Location Domain

The Location Domain is responsible for handling geolocation data, distance calculations, and location-based filtering of vendors and packages. It enables users to discover nearby food offers efficiently.

5.2.7.1 Family of Components

- Geolocation Service (handles coordinate processing and validation)
- Distance Calculation Module (e.g., Haversine formula)
- Geospatial Query Engine (for location-based filtering)
- Location API Layer (REST endpoints for location queries)
- Caching Layer (stores frequently requested nearby results)
- External Map Services Integration (Google Maps)

5.2.7.2 Database Structure

5.2.7.3 Component Relationships

- All application domains interact with the Location Domain via APIs or service

- calls
- Vendor Domain provides static location data for restaurants
- User Domain provides dynamic user location updates
- Order Domain uses location filtering to display relevant packages

5.2.7.4 Data Flow

- User location is obtained via client device (GPS)
- Location data is sent to Location Domain via API
- Location Domain validates and stores coordinates
- Geospatial query is executed to find nearby vendors/packages
- Distance calculations are performed
- Filtered results are returned to application services

5.2.7.5 Error Handling & Edge Cases

- Invalid coordinates → request rejected
- No nearby vendors → empty result
- Location permission denied → fallback behavior

5.2.7.6 Security Considerations

- User location data handled securely
- Location access requires user consent

5.2.7.7 Scalability Considerations

- Uses spatial indexing (e.g., geospatial queries)
- Supports large-scale location queries

5.2.7.8 Dependencies

- Interacts with Vendor Domain (vendor locations)
- Interacts with User Domain (user location data)
- Supports Order Domain (location-based filtering)

5.2.7.9 State Management

- Tracks dynamic user location
- Stores static vendor locations

5.2.7.10 Business Rules

- Only vendors within a certain radius are shown
- Distance-based sorting applied

5.2.7.11 Performance Considerations

- Uses geospatial indexing for fast queries
- Caching nearby results for frequent queries

5.2.7.12 Design Considerations

- High performance for real-time queries
- Privacy-aware design
- Scalable for large geographic datasets

Section 6 – Data Design

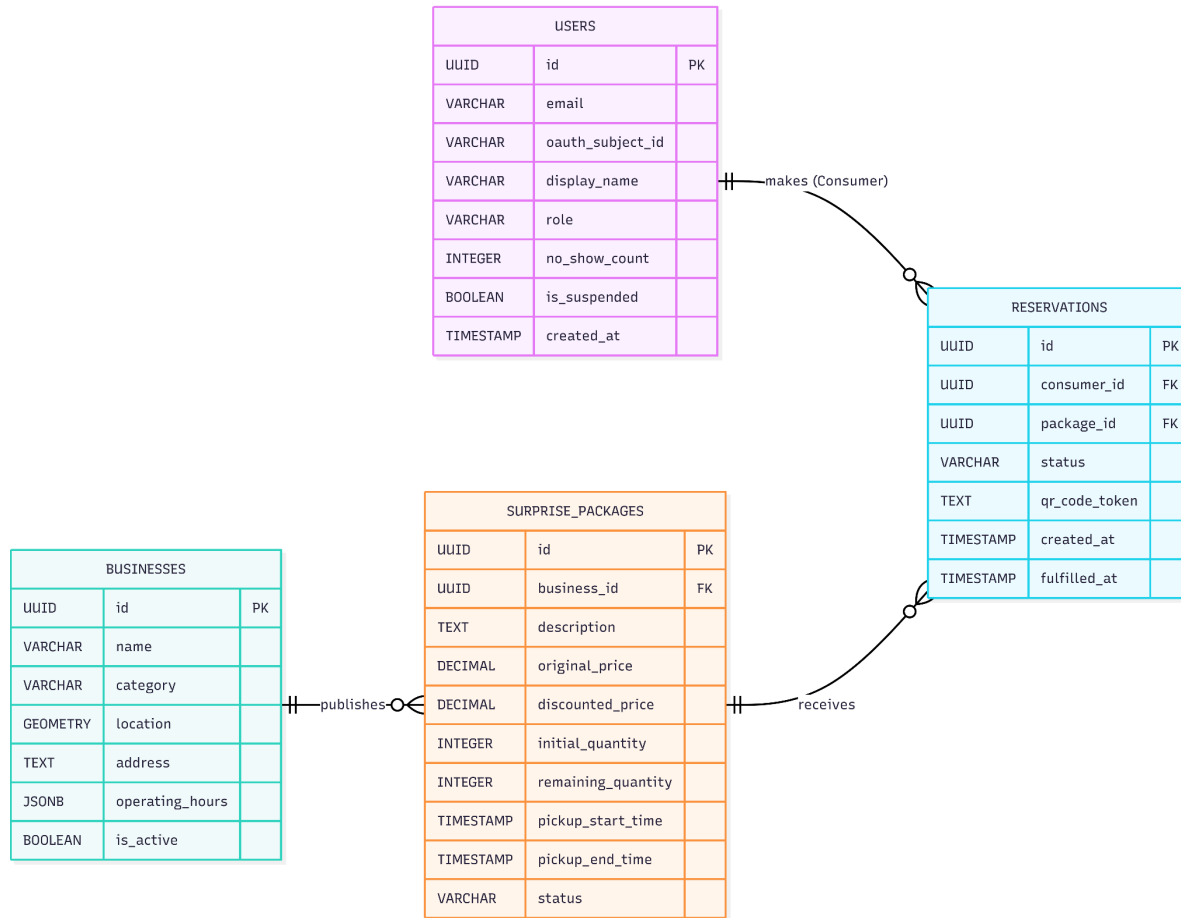
The data design for the Süprüz platform is architected to ensure high performance, strict transactional integrity, and scalable geospatial capabilities. Given the system's fundamental reliance on real-time inventory management and location-based filtering, the data architecture utilizes a centralized PostgreSQL relational database augmented with the PostGIS extension.

This section outlines the structure of both persistent and transient data, mapping the relationships between core system entities (Users, Businesses, Packages, and Reservations). It details the data management strategies implemented to guarantee consistency during high-concurrency flash reservations—specifically utilizing pessimistic locking mechanisms to prevent inventory overselling. Furthermore, it defines the data boundaries and transformation processes between internal application domains and external third-party interfaces, ensuring that sensitive financial and personal data are handled efficiently and securely.

6.1 Persistent/Static Data

The persistent data architecture is built around a highly relational model. The Entity Relationship Diagram (ERD) below illustrates the core tables, their attributes, and the exact constraints that link them together. The schema relies on UUIDs for primary keys to ensure global uniqueness and secure API endpoints.

Figure 6.1 - Core Entity Relationship Diagram



6.1.1 Dataset

The Süprüz dataset is highly relational, centered around the interaction between **USERS** (Consumers) and **BUSINESSES** (Vendors) via **SURPRISE_PACKAGES** and **RESERVATIONS**.

- A **BUSINESSES** record can publish multiple (1:N) **SURPRISE_PACKAGES**.
- A **USERS** record (Role: Consumer) can make multiple (1:N) **RESERVATIONS**.
- A **RESERVATIONS** record acts as a junction entity that explicitly links one USER to one **SURPRISE_PACKAGE** for a specific transaction lifecycle.

6.1.2 Static Data

Static data within the system is minimal but includes predefined enumerations and business rules configured at the database level. This includes User Roles (CONSUMER, STAFF, ADMIN), Package Categories (Bakery, Restaurant, Grocery), and Reservation Statuses (ACTIVE, CLOSED, EXPIRED, NO_SHOW). The system also relies on static geographical boundary logic (the hardcoded 5 KM radius constraint) which is utilized by the PostGIS extension (SRID 4326) during spatial queries.

6.1.3 Persisted data

Persisted data is stored in the centralized PostgreSQL relational database. The most critical persisted data includes the **remaining_quantity** of **SURPRISE_PACKAGES**, which is strictly managed using pessimistic row-level locking (e.g., **SELECT ... FOR UPDATE**) during the reservation flow to guarantee transactional integrity and prevent overselling during high-concurrency flash sales.

6.2 Transient/Dynamic Data

- **Real-time GPS Coordinates:** A consumer's live latitude and longitude are considered highly transient. They are captured by the mobile client, sent via REST API for the immediate spatial query (**ST_DWithin**), and temporarily held in server memory for validation. They are *not* permanently written to the database to preserve user privacy and reduce unnecessary database I/O.
- **JSON Web Tokens (JWT):** Authentication sessions are stateless. JWTs generated upon successful Google/Apple SSO login are stored dynamically on the client device (EncryptedSharedPreferences/Keychain) and held in memory during the lifecycle of an active API request on the backend.
- **In-Memory Caching (Backend):** To optimize performance for the Discovery Screen, frequently queried nearby active packages are temporarily stored in a caching layer (e.g., Spring Cache / Redis). This transient data is invalidated or updated immediately when a package is reserved or closed.
- **Application State (Frontend):** Transient UI states, such as the active countdown timer for QR code expiry, map zoom levels, and category filter selections, are managed dynamically in the React Native client's memory (e.g., Context API / Redux) and are cleared when the app is closed.

6.3 External Interface Data

- **Payment Gateway (iyzico) Payloads:** Financial transaction requests exchanged with the iyzico API are strictly formatted in JSON. The outbound payload includes non-persistent transient data such as **price**, **paidPrice**, **currency** (TRY), **basketItems** (Package ID and Name), and masked **buyer** details. The inbound response data contains the **paymentStatus** and an external **transactionId**. No credit card pan numbers are routed through or stored in the Süprüz database.
- **Push Notification (Expo/FCM/APNs) Payloads:** Ephemeral JSON payloads are dispatched to Expo's Push API. The data structure includes a **to** (device push token), **title**, **body**, and a **data** object containing routing metadata (e.g., **{"screen": "ReservationDetail", "id": "uuid"}**) to enable deep-linking when the user taps the notification.
- **Mapping Services (Google Maps API):** External UI data fetched directly by the mobile client. This includes map vector tiles, point-of-interest (POI) rendering data, and calculated routing distances, all of which remain transient and are rendered strictly on the client side.

6.4 Transformation of Data

- **DTO to Entity Mapping:** As data moves through the Spring Boot backend, external JSON payloads from REST API requests are deserialized into Data Transfer Objects (DTOs) for validation. Before persistence, these DTOs are transformed into JPA Entities. Conversely, Entities are transformed back into response DTOs to ensure sensitive database fields (like hidden IDs or audit timestamps) are not leaked to the client.

- **Geospatial Transformation:** Standard numeric GPS coordinates (Latitude: 38.4237, Longitude: 27.1428) received from the mobile client are transformed by the backend into specialized PostGIS geometry objects using the spatial function `ST_SetSRID(ST_MakePoint(lon, lat), 4326)`. This transformation is mandatory for rapid, indexed radius calculations.
- **Cryptographic Data Signing (QR Codes):** To prevent QR fraud, relational data (Reservation ID, Business ID, and Expiry Timestamp) is aggregated and transformed via a cryptographic hashing algorithm (HMAC-SHA256) using a secret server key. This transforms readable data into a secure, encrypted token string that the Business Portal decrypts and verifies during the in-store scan.
- **State Machine Transitions:** Data representing the state of a reservation undergoes logical transformations governed by business rules (e.g., transitioning from **RESERVED** to **VERIFIED/PENDING PAYMENT**, and finally to **COMPLETED** based on sequential API triggers from the QR scanner and POS confirmation).

Section 7 - User Interface Design

7.1 User Interface Design Overview

The Süprüz user interface is designed with a "Forest-to-Night" aesthetic, prioritizing visual excellence, fluid motion, and intuitive interactions. Built using React Native and Expo, the application follows a modern design system that ensures consistency across both iOS and Android platforms.

7.1.1 Design Principles

Vibrant & Eco-Friendly Palette: Utilizing a curated selection of forest greens, teals, and lime accents to reflect the sustainability mission.

Dark Mode Support: A comprehensive dark theme (Night) is implemented to reduce eye strain and provide a premium, high-contrast experience.

Micro-Animations: Integration of React Native Reanimated for smooth transitions, such as bottom-sheet overlays, button scaling, and list entrance animations.

Accessibility: Adherence to WCAG 2.1 Level AA standards, ensuring touch targets are ≥ 44 pt and color contrast ratios are optimized for readability.

7.1.2 Key UI Components

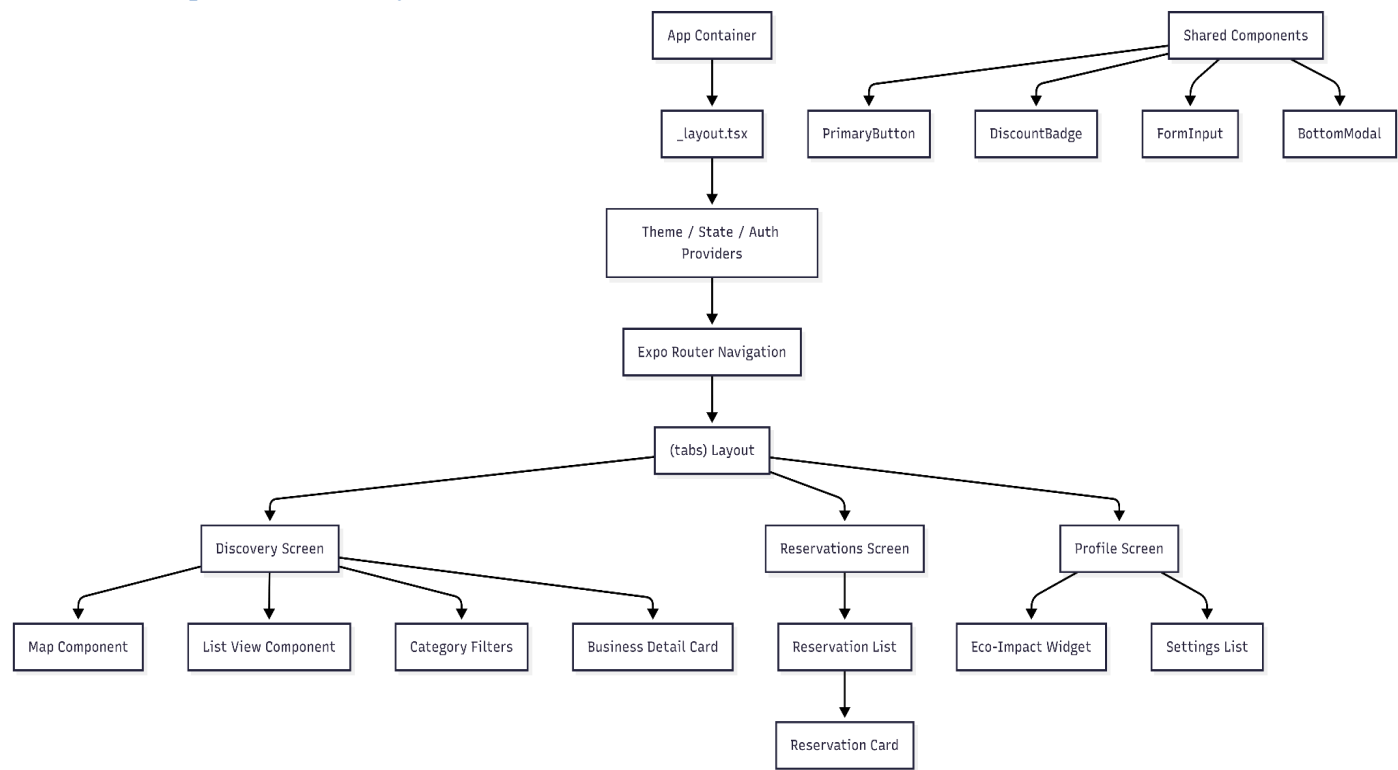
Onboarding Carousel: A 4-slide interactive introduction using smooth horizontal paging and animated icons to educate users on the "Search, Reserve, Pick-up" flow.

Discovery Map: A customized map implementation featuring branded pins and a real-time proximity geofence indicator.

Business Detail Bottom-Sheet: A non-intrusive modal that provides quick access to package details, countdown timers, and the "Reserve" action.

Eco-Impact Dashboard: A visual representation of the user's contribution to sustainability (CO2 saved, meals saved, money saved) using modern glassmorphism cards.

Technical Component Hierarchy:



7.2 User Interface Navigation Flow

The application utilizes a file-based navigation system to separate the Consumer and Business experiences. The navigation logic is focused on minimizing taps and providing immediate access to core tools.

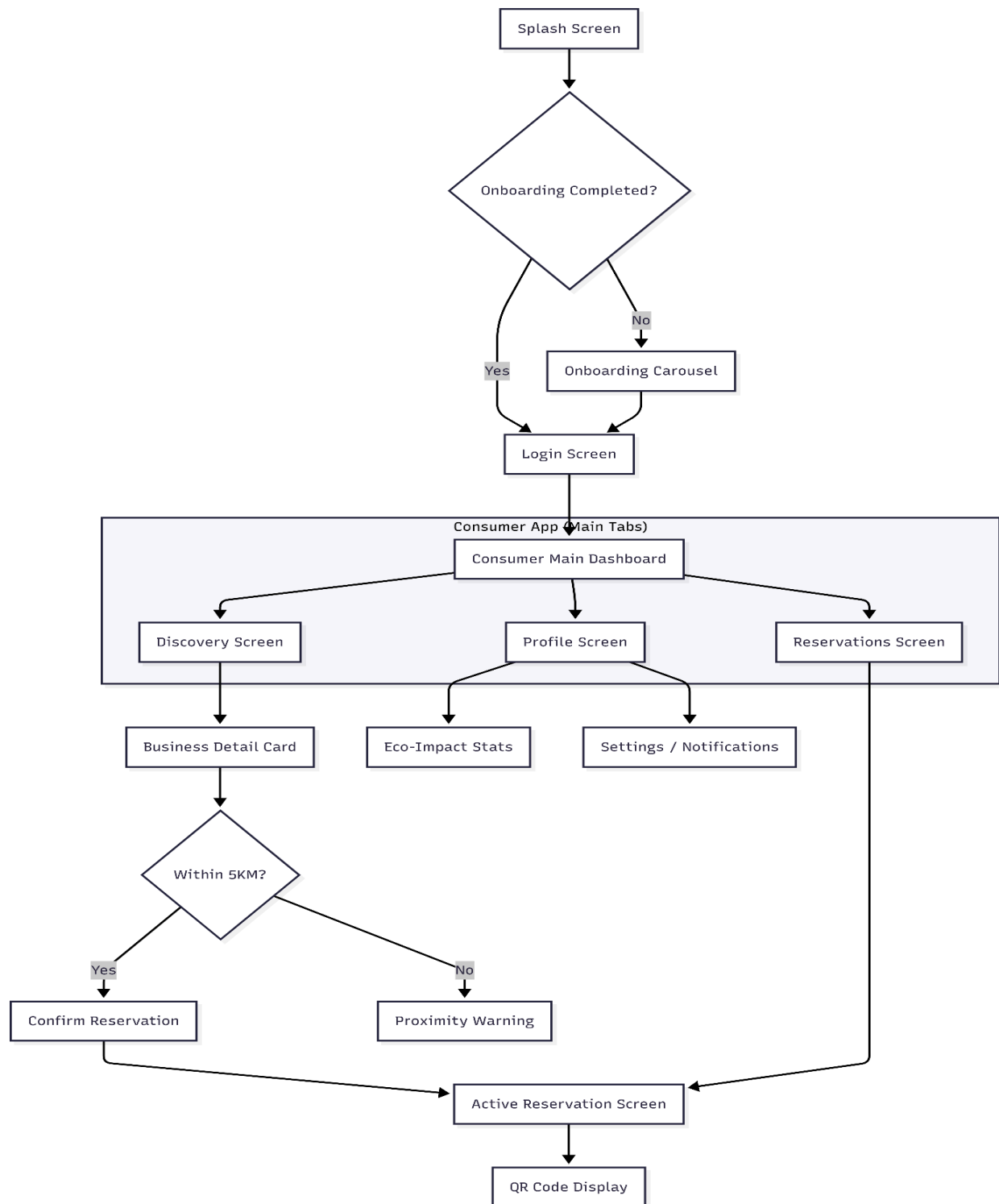
7.2.1 Consumer Navigation Path

Primary Navigation: Managed via a persistent bottom tab bar (Discover, Reservations, Profile).

Interaction Flow: Tapping a map pin highlights the business, and tapping the preview card transitions to the detailed view.

Gestures: The Discovery Map supports standard pinch-to-zoom and pan gestures for location-based exploration.

Consumer Navigation Flow



7.2.2 Business Partner Navigation Path

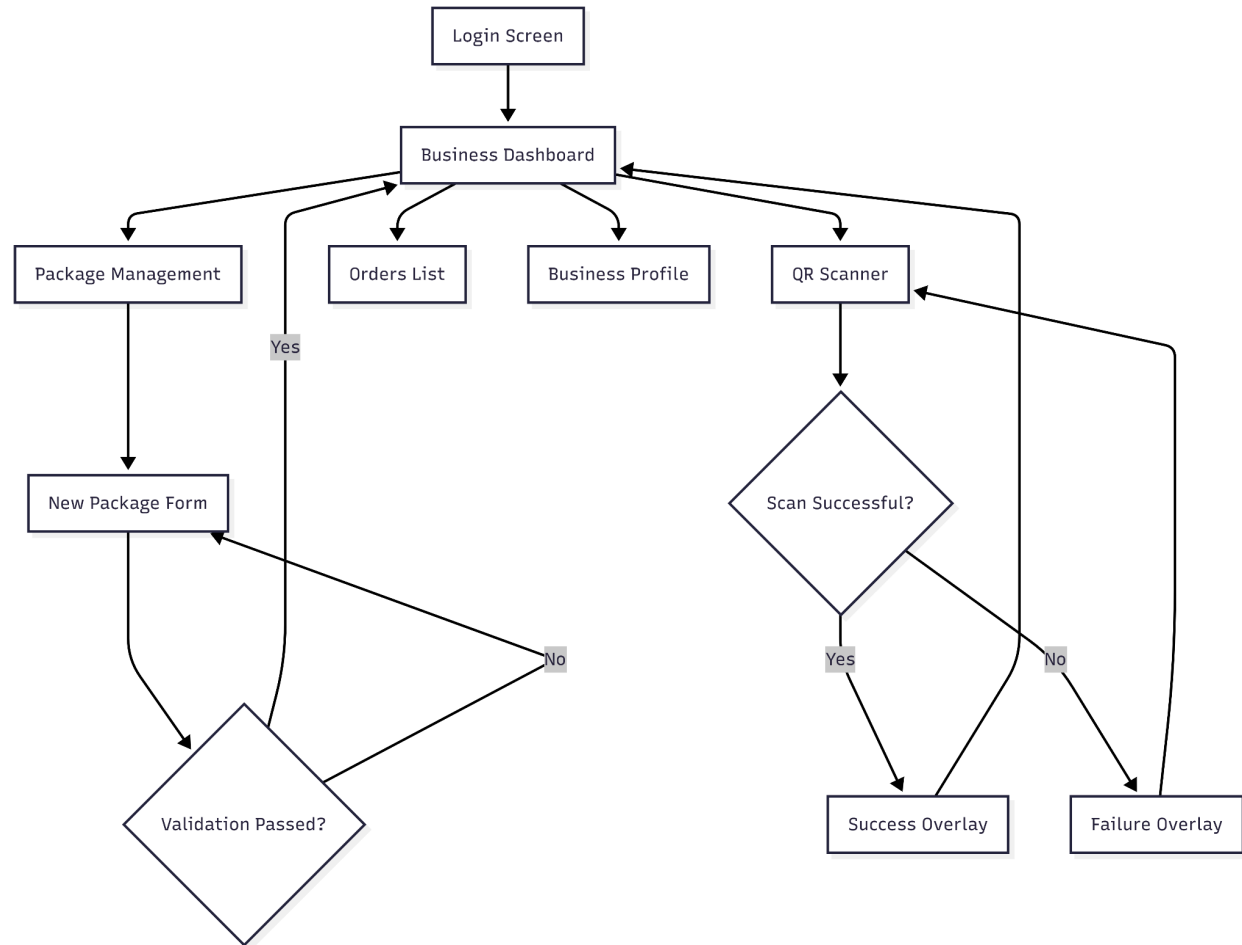
Primary Navigation: A flat tab-based structure (Dashboard, Packages, Scanner, History) allows staff to

switch tasks instantly.

Modals: Inventory creation and editing are handled via full-screen vertical modals to ensure focus during data entry.

Real-Time Preservation: Navigation states are preserved during auto-refreshes of inventory data.

Business Navigation Flow



7.3 Use Cases / User Function Description

7.3.1 UC-01: Discovering Nearby Packages

Description: The user finds available surplus food listings within their immediate vicinity.

Functionality:

The system requests GPS permissions and centers the map on the user.

Category filters (Bakery, Restaurant, Grocery) allow for rapid narrowing of results.

A toggle switch allows swapping between interactive Map View and distance-sorted List View.

7.3.2 UC-02: Proximity-Based Reservation

Description: The user reserves a "Surprise Package" from a local business.

Functionality:

The "Reserve" button is dynamically enabled only if the user is within a 5KM radius. If outside the radius, a proximity warning banner informs the user of the distance requirement. Upon tap, the system performs a final server-side location validation before confirming the inventory lock.

7.3.3 UC-03: QR-Code Fulfillment

Description: The user verifies their reservation at the business location to pick up their package.

Functionality:

A high-contrast QR code is generated for the active reservation.
A real-time countdown timer shows the remaining time in the pickup window.
Status badges update in real-time once the business scans the code.

7.4 Business Partner Interface Design

The Business Partner interface is optimized for high-speed operation in retail environments, focusing on inventory management and rapid verification.

7.4.1 Key Business Screens Breakdown

7.4.1.1 Business Dashboard (Operational Hub)

Description: The primary status screen for managing daily pickup traffic.

Layout / Elements: Global KPI widgets (Revenue, Active Listings), scrollable "Active Package" list, and Floating Action Button (FAB).

Inputs: Tap on FAB (Navigates to Create), Tap on Card (Navigates to Edit).

Outputs / Responses: Real-time refresh of reservation counts every 30 seconds via socket or polling.

7.4.1.2 Package Creation Form

Description: A data-entry utility for listing surplus inventory.

Layout / Elements: Text inputs for description, numeric fields for pricing/stock, and native time-pickers for pickup windows.

Inputs: Title, Original Price (฿), Discounted Price (฿), Quantity, Start/End Times.

Outputs / Responses: Live validation indicator. Fields highlight red if the discount is < 50% or the window is < 30 minutes.

7.4.1.3 QR Scanner Interface

Description: The transaction terminal for reservation fulfillment.

Layout / Elements: Full-screen live camera viewfinder with a semi-transparent target grid overlay.

Inputs: Optical scan of consumer QR code.

Outputs / Responses: Haptic pulse and opaque state overlay (Green for "Verified", Red for "Invalid/Expired").

7.4.1.4 Order Ledger (History)

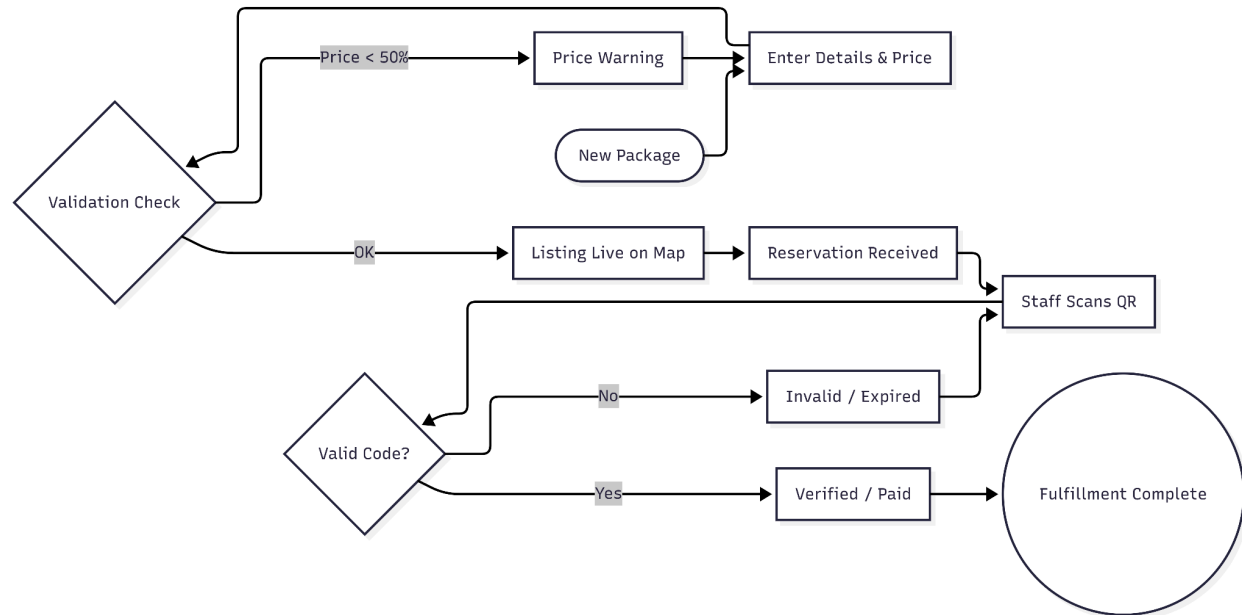
Description: A historical record of all fulfilled and missed pickups.

Layout / Elements: Searchable table containing Customer Name, Time of Scan, and Status Pill.

Inputs: Text search and status filtering (Completed vs No-Show).

Outputs / Responses: Filtered list of past transactions.

Business Fulfillment Lifecycle



7.4.2 Business User Experience

Efficiency: The QR verification flow is designed to complete in under 2 seconds to minimize friction at the checkout counter.

Persistence: The dashboard auto-refreshes every 30 seconds to ensure staff are always viewing the most current inventory levels.

Section 8 - Other Interfaces

This section describes the external interfaces utilized by the Süprüz system. These interfaces enable integration with third-party services for authentication, geolocation, payment processing, and push notifications. All communications are conducted over secure protocols and follow standardized data formats.

8.1 Push Notification Interface (Expo Push Service)

Purpose:

To deliver real-time notifications to consumer devices regarding new packages, reservation updates, and reminders.

Technology:

Expo Push Notification Service (built on Firebase Cloud Messaging for Android and Apple Push Notification Service for iOS)

Protocol:

HTTPS

Data Format:

JSON payload

Interaction Flow:

1. Backend generates a notification event (e.g., new package published).
2. Backend sends a request to the Expo Push API.
3. Expo routes the message to:
 - Firebase Cloud Messaging (Android)
 - Apple Push Notification Service (iOS)
4. Notification is delivered to the user's device.

Failure Conditions:

- Notification delivery may fail due to network issues.
- Notifications are silently dropped if the user has disabled permissions.
- Invalid or expired push tokens result in failed delivery.

Notes:

Expo provides a unified interface for cross-platform notification delivery, reducing platform-specific complexity.

8.2 Geolocation and Mapping Interface (Google Maps API)

Purpose:

To provide location services, map visualization, and nearby business discovery.

Technology:

Google Maps Platform APIs

Protocol:

HTTPS (RESTful API calls)

Data Format:

JSON

Interaction Flow:

1. Client retrieves device GPS coordinates.

2. Coordinates are sent to the backend.
3. Backend performs spatial queries using PostGIS.
4. Results are optionally enriched or visualized using Google Maps services.

Failure Conditions:

- API unavailability results in degraded functionality (list-only view).
- Inaccurate GPS data may affect discovery accuracy.

Security:

- API key-based authentication.

8.3 Payment Gateway Interface (iyzico)

Purpose:

To securely process in-app payments for reservations.

Technology:

iyzico Payment API

Protocol:

HTTPS

Data Format:

JSON

Interaction Flow:

1. Backend sends payment request with transaction details.
2. iyzico processes the payment securely.
3. Response (success/failure) is returned to backend.
4. Reservation status is updated accordingly.

Failure Conditions:

- Payment failure due to insufficient funds or authorization errors.
- Network issues may interrupt transaction processing.

Security:

- Sensitive payment data is not stored within the system.
- All communication is encrypted via HTTPS.

8.4 Authentication Interface (OAuth Providers – Google & Apple)

Purpose:

To enable secure user authentication via third-party providers.

Technology:

OAuth 2.0

Protocol:

HTTPS

Data Format:

JSON (tokens and user info)

Interaction Flow:

1. User selects an authentication provider.
2. Client redirects to provider's login page.
3. User authenticates and grants permission.
4. Provider returns an authorization token.
5. Backend validates token and creates or logs in user.

Failure Conditions:

- Invalid or expired tokens result in authentication failure.
- User denial of permissions cancels login process.

Security:

- Token-based authentication.
- No password handling required for SSO users.

8.5 QR Code Generation and Verification Interface

Purpose:

To enable secure reservation verification during in-store pickup.

Technology:

Custom QR generation and scanning module

Protocol:

Internal system communication over HTTPS

Data Format:

Encoded and encrypted data within QR code

Interaction Flow:

1. Backend generates a unique, time-limited QR code after reservation.
2. Consumer presents QR code at the business location.
3. Business portal scans the QR code.
4. Backend validates:
 - Reservation ID
 - Timestamp validity
 - Business association

5. Reservation status is updated.

Failure Conditions:

- Expired QR code (pickup window passed).
- QR code already used (duplicate scan).
- QR code associated with a different business.

Security:

- QR codes are cryptographically signed.
- Time-based validity prevents replay attacks.

8.6 REST API Interface (Client–Backend Communication)

Purpose:

To enable communication between client applications and backend services.

Technology:

RESTful Web Services (Spring Boot)

Protocol:

HTTPS

Data Format:

JSON

Interaction Flow:

- Clients send HTTP requests (GET, POST, PUT, DELETE).
- Backend processes requests and returns structured responses.

Failure Conditions:

- Network errors or server downtime.
- Invalid requests return appropriate HTTP status codes (e.g., 400, 401, 500).

Security:

- HTTPS encryption
- Token-based authentication (JWT or session-based)

Section 9 - Extra Design Features / Outstanding Issues

Does not fit anywhere else above, but should be mentioned -- goes here

Section 10 – References

[SRS Document](#) - Fits exactly with this document in each version.

Section 11 – Glossary

Glossary of terms / acronyms